

Assignment 5

Asher Cronin (axc1541)

1

Given the list [12, 5, 9, 3, 15, 7, 2, 10, 6, 8] show each step of a sorting method algorithm.

1.a Merge Sort

```
[12, 5, 9, 3, 15, 7, 2, 10, 6, 8]
[12, 5, 9, 3, 15] [7, 2, 10, 6, 8]
[12, 5, 9] [3, 15] [7, 2, 10] [6, 8]
[12, 5] [9] [3] [15] [7, 2] [10] [6] [8]
[12] [5] [9] [3] [15] [7] [2] [10] [6] [8]
[5, 12] [9] [3, 15] [2, 7] [10] [6, 8]
[5, 9, 12] [3, 15] [2, 7, 10] [6, 8]
[3, 5, 9, 12, 15] [2, 6, 7, 8, 10]
[2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
```

1.b Quick Sort (I will pick the last element as pivot)

```
[12, 5, 9, 3, 15, 7, 2, 10, 6, 8]
[5, 3, 7, 2, 6, 8, 12, 10, 15, 9]
[5, 3, 7, 2, 6] 8 [12, 10, 15, 9]
[5, 3, 2, 6, 7] 8 [9, 10, 15, 12]
[5, 3, 2] 6 [7] 8 9 [10, 15, 12]
[2, 3, 5] 6 7 8 9 [10, 12, 15]
  2 [3, 5] 6 7 8 9 [10] 12 [15]
  2 [3] 5 6 7 8 9 10 12 15
  2 3 5 6 7 8 9 10 12 15
→ [2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
```

1.c Selection Sort

```
[12, 5, 9, 3, 15, 7, 2, 10, 6, 8]
[2, 5, 9, 3, 15, 7, 12, 10, 6, 8]
[2, 3, 9, 5, 15, 7, 12, 10, 6, 8]
[2, 3, 5, 9, 15, 7, 12, 10, 6, 8]
[2, 3, 5, 6, 15, 7, 12, 10, 9, 8]
[2, 3, 5, 6, 7, 15, 12, 10, 9, 8]
[2, 3, 5, 6, 7, 8, 12, 10, 9, 15]
[2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
[2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
[2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
[2, 3, 5, 6, 7, 8, 9, 10, 12, 15]
```

2

2.a Detailed Bubble Sort ([34, 7, 23, 32, 5, 62])

The list starts as [34, 7, 23, 32, 5, 62] which is visibly unordered, but the computer doesn't know that, so it will just run the given algorithm.

In this case the algorithm is bubble sort.

Bubble Sort works in passes:

Pass 1

[34, 7, 23, 32, 5, 62] → [7, 23, 32, 5, 34, 62]

The first two items are compared and swapped if necessary (34 and 7 are swapped).

Then the next two items (now 34 and 23), and then the next, all the way until the biggest number reaches the end of the array.

Pass 2

[7, 23, 32, 5, 34, 62] → [7, 23, 5, 32, 34, 62]

The same thing happens in this pass, though only 32 and 5 needed to be switched.

Pass 3

[7, 23, 5, 32, 34, 62] → [7, 5, 23, 32, 34, 62]

Here 5 and 23 got swapped.

Pass 4

[7, 5, 23, 32, 34, 62] → [5, 7, 23, 32, 34, 62]

In this pass 5 and 7 switched, and we can see that the array is now sorted, but the algorithm will have to do a full pass with now swaps to verify that it is sorted.

Pass 5

The algorithm sees that the array is now sorted and returns the array

2.b Time Complexity of Bubble Sort

2.b.a Best Case

$O(N)$, the array is already sorted and it just has to go through the array once and check that all the items are in the right place.

2.b.b Average Case

$O(N^2)$, the array is jumbled relatively randomly. Each time the algorithm goes through the array (or N comparisons) at least one more item gets sorted into place at the end of the array, so the maximum number of passes would be the length of the array or N .

2.b.c Worst Case

$O(N^2)$, the array is sorted in reverse and it takes N passes of N comparisons and N swaps.

3

3.a Detailed Bucket Sort

The initial array is: [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]

We are using 5 buckets, so the items will be multiplied by 5 and the corresponding integer will be the index for each item. (item · 5 % 5)

The array of table indeces for the array is: [3,0,1,1,3,4,1,0,1,3]

Put the items in the bucket according to their index and join then using a linked list.

The table is as follows

0	0.17 → 0.12
1	0.39 → 0.26 → 0.21 → 0.23
2	
3	0.78 → 0.72 → 0.68
4	0.92

Sort the linked lists using any method (table as follows):

0	0.12 → 0.17
1	0.21 → 0.23 → 0.26 → 0.39
2	
3	0.68 → 0.72 → 0.78
4	0.92

Put back into array from buckets, in the order of the bucket indeces and in the order within each bucket.

→ [0.12, 0.17, 0.21, 0.23, 0.26, 0.39, 0.68, 0.72, 0.78, 0.92]

3.b Number of buckets on performance

Increasing the number of buckets makes it more likely that each individual bucket won't need to be sorted which saves on time complexity but increases memory complexity, and decreasing the number of buckets does the exact opposite. The number of buckets is a tradeoff between the time it takes sorting each bucket and memory.

3.c Uneven Distribution

This greatly impacts the effectiveness of the bucket sort, and an uneven dataset is very inefficient. One way to make the distribution more even is to change the hashing method, or place the items into buckets in a different way, or have a dynamic bucket allocation and provide more buckets for areas with more data and less for scarcer areas.

3.d Time Complexity

The time complexity of Bucket Sort is $O(N+K)$ where N is the size of the original array and K is the size of the longest linked lists in the buckets.

Bucket Sort would be a better choice than Quick Sort or Merge sort if the program needs to be parallelized (this is quite easy with bucket sort because it involves almost no comparisons), or if the dataset is relatively small and uniform.

4 Implementation of Insertion Sort

sort (array)

- for i = 1 until i < array length
- - key = arr[i]
- - j = i-1
- - while arr[j] > key
- - - move arr[j] up and move j down

j delineates the sorted portion of the array, and the key is the item to add to that. If the item is in the right place already do nothing, if it is less than the last one put it before that one and keep moving it upwards until its in the right place.

Best case: $O(N)$

Average case: $O(N^2)$ Worst case: $O(N^2)$